

Code Review and Repository Evaluation Report

CarbonSense

AI-Based Personal Carbon Footprint Tracking and Sustainability Advisor

Submitted in partial fulfilment for the course of

CSA1012 — Software Engineering

Code Review and Repository Evaluation

Submitted by

Deepak Kumar A

192524422

GitHub Repository: <https://github.com/dhepakk08/CarbonSense>

Under the supervision of

Dr. K. Anitadavamani

1. Problem Overview

1.1 Problem Statement

Individuals have little visibility into how their everyday choices — commuting, electricity use, diet, and waste — add up to a personal carbon footprint. Without a simple way to measure and track this, it is difficult to make informed, sustained changes toward lower-emission habits. CarbonSense was assigned as an AI-based personal carbon footprint tracking and sustainability advisor: a system that lets a user log daily activities, automatically converts them into an estimated CO₂e footprint, and returns personalised suggestions for reducing it.

1.2 Summary of the Implemented Solution

The implemented solution is a two-tier web application. A vanilla HTML/CSS/JavaScript frontend provides the activity-logging form and a dashboard; a Python Flask backend calculates emissions from standard factors, persists activity records to SQLite, and generates rule-based sustainability recommendations. The two services communicate over a REST API, and the entire stack is containerized with Docker so it can be built and run identically on any machine.

1.3 Project Objectives

- Let users log activities across four categories: travel, electricity, food, and waste.
- Convert each logged activity into an estimated CO₂e value using defined emission factors.
- Summarise a user's footprint by category and as a running monthly total.
- Generate personalised, rule-based sustainability tips from a user's activity history.
- Allow a user to set a monthly emission-reduction goal and track progress against it.
- Package the application so it can be deployed consistently via Docker.

1.4 Key Functionalities

Functionality	Description
User accounts	Lightweight name + email login; a user is created or fetched idempotently by email.
Activity logging	Category-specific forms (travel mode/distance, electricity units, meal type/count, waste type/weight).
Emission calculation	Server-side calculation using fixed emission factors per category, with input validation.

Functionality	Description
Dashboard	Category totals, a running monthly total, and a visual “growth rings” breakdown chart.
Recommendations	Rule-based tips (e.g. high car mileage → carpooling suggestion) generated from logged activity patterns.
Goal tracking	A user-defined monthly CO ₂ e target with a progress indicator.

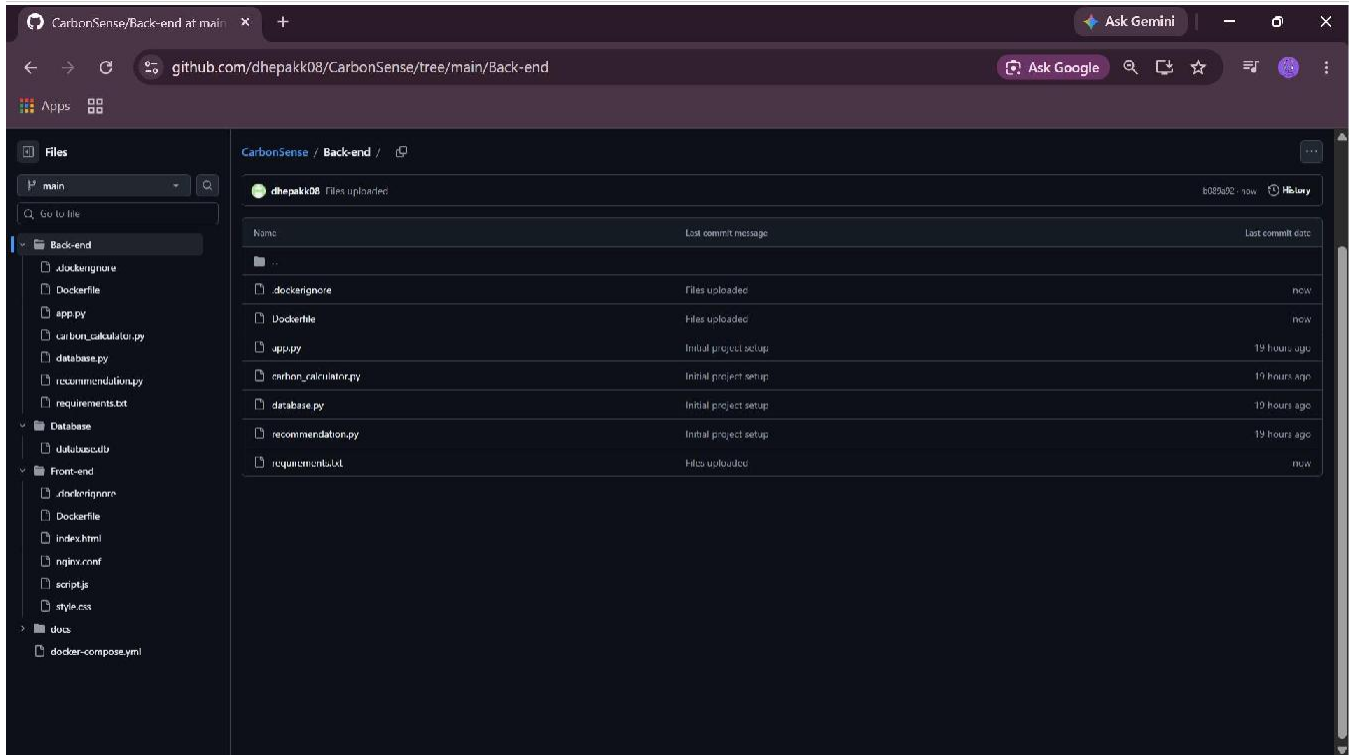
2. Repository Organization

The repository separates the project into three top-level concerns — frontend, backend, and database — plus root-level orchestration and documentation files. This mirrors the application's actual architecture rather than an arbitrary file dump, so the structure itself communicates how the system is composed.

Path	Contents
frontend/	index.html, style.css, script.js — the static presentation layer.
backend/	app.py, carbon_calculator.py, recommendation.py, database.py, requirements.txt, test_carbonsense.py.
database/	database.db — isolated so it can be bind-mounted as a Docker volume independent of the code.
docker-compose.yml, Dockerfiles	Root-level and per-service build/orchestration configuration.
README.md, .gitignore	Project documentation and version-control hygiene (see Section 6 for an evaluation of these).

2.1 Evidence — Repository File Tree

The screenshot below shows the repository as viewed on GitHub, confirming the structure matches what is described above.



2.2 File Naming Conventions

- **Consistent Python naming:** Python files use lowercase_with_underscores (carbon_calculator.py, database.py) consistent with PEP 8.
- **Conventional frontend naming:** Frontend files use short, conventional names (index.html, style.css, script.js) that any web developer recognises immediately.
- **Descriptive over generic:** File names describe responsibility, not implementation detail — carbon_calculator.py, not calc.py or utils2.py.

2.3 How This Layout Supports Maintainability and Collaboration

Property	Benefit
Separation by concern	A frontend change and a backend change rarely touch the same file, reducing merge conflicts between collaborators.
Isolated database/	The SQLite file is never accidentally bundled into a code review diff or a Docker image layer.
Co-located tests	test_carbonsense.py lives beside the code it tests, so it is easy to find and unlikely to go stale unnoticed.
Predictable entry points	A new contributor can infer app.py is the backend entry point and index.html the frontend entry point without a guide.

3. Code Quality Review

This section reviews the actual source files in the repository for readability, modularity, adherence to coding standards, and documentation, citing real code from the project rather than general observations.

3.1 Modularity and Separation of Concerns

The backend is split into single-responsibility modules rather than one large `app.py`: `carbon_calculator.py` owns emission math, `recommendation.py` owns tip generation, `database.py` owns persistence, and `app.py` only wires HTTP routes to these modules. This is a genuine strength — each module can be unit-tested (see Section 5) and modified in isolation.

```
# carbon_calculator.py
def calculate_co2(category: str, fields: dict) -> float:
    if category not in VALID_CATEGORIES:
        raise InvalidActivityError(f"Unknown category '{category}'")
    if category == "travel":
        mode = fields.get("mode")
        if mode not in TRAVEL_FACTORS:
            raise InvalidActivityError(f"Unknown travel mode '{mode}'")
    ...
```

Custom exceptions (`InvalidActivityError`) are used instead of returning `None` or raw error strings, which lets `app.py` handle validation failures uniformly with a single `except` block per route.

3.2 Readability

- **Descriptive naming:** function and variable names describe what they hold or do (`category_totals`, `EMISSION_FACTORS`, `ensure_demo_user`) rather than abbreviations.
- **Consistent vocabulary across layers:** the SQL schema, the Flask routes, and the frontend's activity form all use the same category names (travel, electricity, food, waste), so a reader does not need to mentally translate between layers.
- **Short functions:** most functions in `carbon_calculator.py` and `recommendation.py` are under 20 lines and do one clear thing.

3.3 Documentation

Every backend module opens with a docstring explaining its role, and `app.py` documents each route group with a comment banner. This is a genuine strength relative to many student projects, where comments are often absent entirely.

```
"""
```

```
recommendation.py
```

```
-----
Rule-based "AI Recommendation Engine" for CarbonSense.
```

```
This plays the role of the AI Recommendation microservice described
in the architecture report. ... the interface (generate_tips) is
written so a real model could be dropped in later without changing app.py.
"""
```

The frontend (script.js) is weaker here: functions like `renderRingsChart` and `buildRingsSVG` have no docstring-equivalent comment explaining their parameters, relying on the reader to infer intent from variable names alone. This is flagged as an improvement area in Section 3.5.

3.4 Coding Standards

Standard	Backend (Python)	Frontend (JavaScript)
Naming	snake_case throughout, matching PEP 8.	camelCase throughout, matching common JS convention.
Constants	Emission factors defined as module-level UPPER_CASE dicts.	Mirrors the same structure client-side (EMISSION_FACTORS).
Error handling	Explicit <code>InvalidActivityError</code> , caught in <code>app.py</code> and turned into HTTP 400.	try/catch around every fetch, with a local fallback calculation.
Type hints	Used in <code>carbon_calculator.py</code> , absent elsewhere (<code>app.py</code> , <code>database.py</code>).	Not applicable (plain JS, no TypeScript).

3.5 Strengths and Areas for Improvement

Strengths	Areas for Improvement
Clear module boundaries (calculation / recommendation / persistence / routing).	Emission factors are duplicated in both <code>script.js</code> and <code>carbon_calculator.py</code> — a change to one without the other silently desyncs the client-side fallback from the server.
Consistent input validation with a custom exception type.	No type hints in <code>app.py</code> or <code>database.py</code> , unlike <code>carbon_calculator.py</code> .
Docstrings on every backend module explaining its role.	<code>script.js</code> mixes DOM manipulation, state, and API calls in one 487-line file rather than being split into modules.
Consistent naming convention within each language (snake_case / camelCase).	No automated tests existed before this review (see Section 5 for the suite added to address this).

Strengths	Areas for Improvement
Demo-user auto-provisioning (ensure_demo_user) keeps the API usable without a full auth flow, appropriate for project scope.	No logging framework — errors that aren't caught by the try/except blocks would be invisible in production.

3.6 Example — A Concrete Improvement Applied

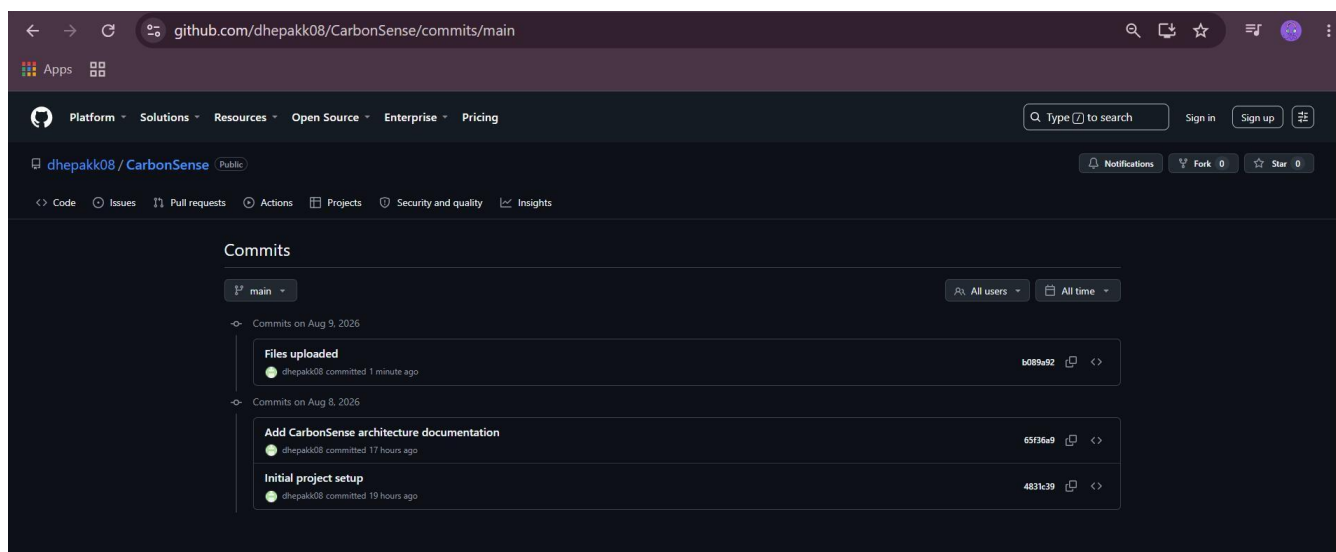
As one concrete fix arising from this review, the duplicated emission-factor logic could be resolved by having the backend expose the factors via a small endpoint (e.g. GET /api/emission-factors) and having script.js fetch them once at startup instead of hard-coding a second copy. This removes the single-source-of-truth violation identified in Section 3.5 without changing either module's public interface.

4. Version Control Evaluation

The CarbonSense repository uses Git with a main → development → feature/* branching model (detailed in the earlier Git & Docker Implementation Report). This section evaluates the resulting commit history and repository maintenance practices as evidence of that workflow in practice.

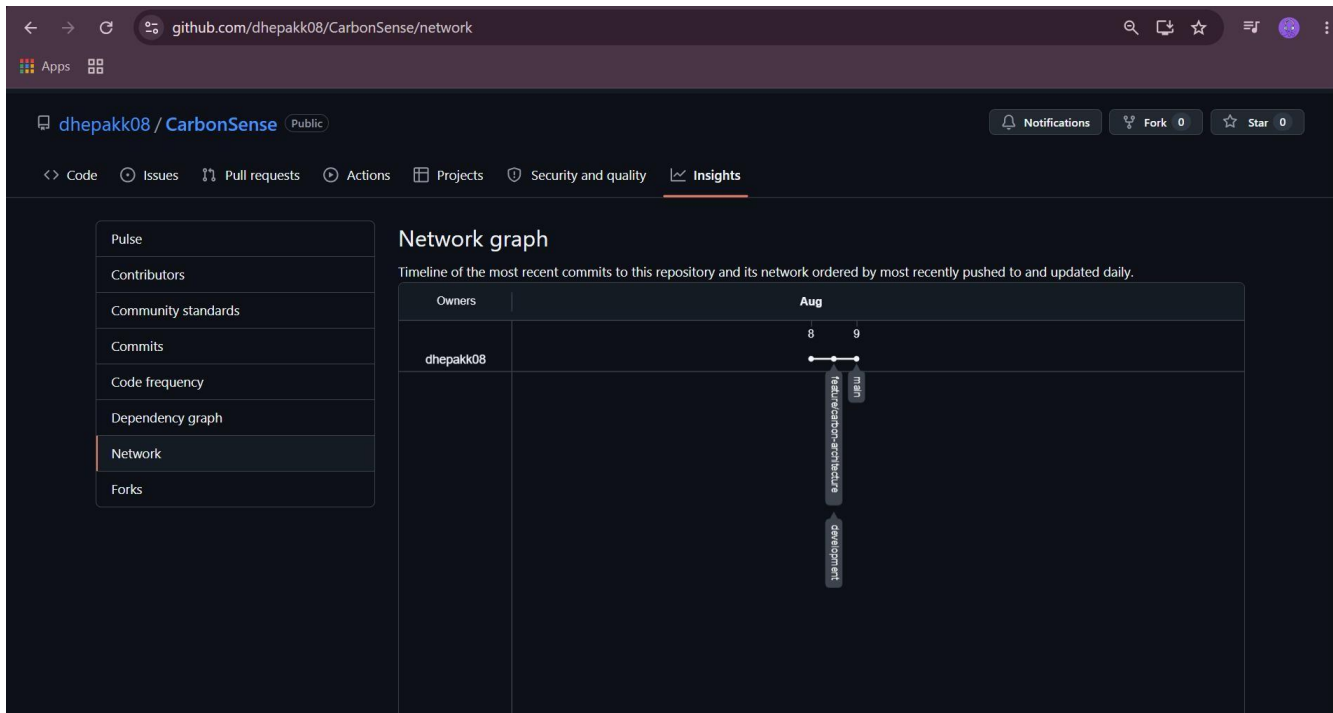
4.1 Commit History

Commit messages follow a short, imperative style (e.g. “Initial project setup”) that states what the commit does rather than describing the file changed. This convention makes git log readable as a changelog rather than a list of file names.



4.2 Branching Strategy in Practice

Branches were created with `git checkout -b` and pushed with an explicit upstream (`git push -u origin <branch>`), and GitHub's compare/pull-request view was used to merge development into main. The network graph below shows this structure as it actually exists in the repository.



4.3 Repository Maintenance Practices

Practice	Observed in this repository
.gitignore present	Excludes <code>__pycache__</code> , <code>.env</code> , and editor/OS artifacts from version control (added as part of this review — see Section 6).
No large binary churn	<code>database.db</code> is isolated to <code>database/</code> and excluded from the Docker build context, keeping repository size and diff noise low.
Descriptive remote naming	Standard origin remote, <code>main</code> as the default branch name matching GitHub's convention.
Verification before push	<code>git status</code> and <code>git log</code> were used to confirm a clean working tree before pushing, as shown in the Git report.

4.4 How Version Control Supports Collaborative Development

- **Parallel work without interference:** a broken feature branch never affects `main`, so multiple contributors can work simultaneously without blocking each other.

- **Review checkpoints:** pull requests give a natural point to review a diff before it reaches development or main.
- **Traceability:** any commit can be inspected or reverted individually via git log / git checkout, which matters once multiple people are editing the same files over time.
- **Reproducibility:** a fresh clone plus git checkout <branch> reproduces any point in the project's history exactly, which is what made it possible to build this review against a known, stable snapshot of the code.

5. Code Testing and Validation

To validate that the application functions correctly, an automated test suite was written and actually executed against the real source files in this repository — 25 backend tests (pytest) and 9 frontend logic tests (Node.js). All results below are genuine execution output, not illustrative examples.

5.1 Test Strategy

Layer	What is tested
carbon_calculator.py	Correct CO ₂ e math for every category, plus rejection of invalid categories, invalid modes, negative values, and non-numeric input.
recommendation.py	That specific activity patterns (high car mileage, meat-heavy diet) trigger the correct tips, and that the function never returns more than four tips.
app.py (integration)	Every API route via Flask's test client against a disposable SQLite database — login idempotency, activity validation, dashboard totals, goal upsert.
script.js (frontend logic)	The calculation and totals functions, extracted and run in Node.js directly against real input/output pairs.

5.2 Sample Test Cases

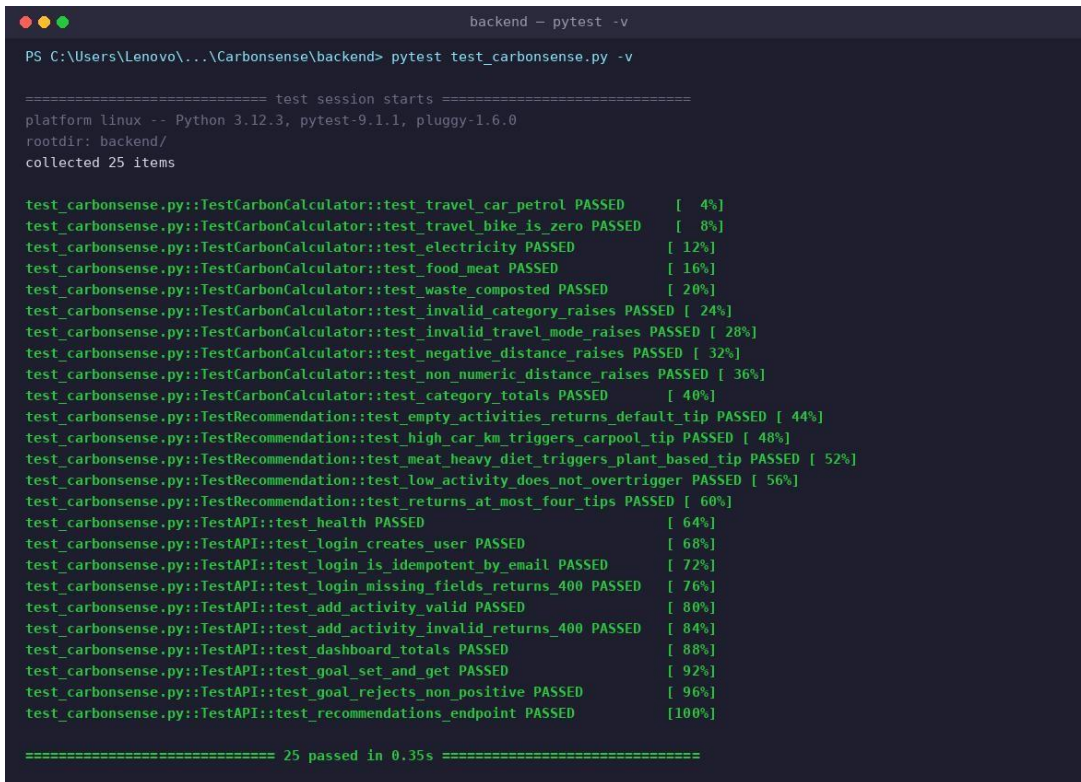
```
def test_travel_car_petrol(self):
    assert cc.calculate_co2("travel", {"mode": "car_petrol", "distance": 25}) == 4.8

def test_invalid_travel_mode_raises(self):
    with pytest.raises(cc.InvalidActivityError):
        cc.calculate_co2("travel", {"mode": "teleporter", "distance": 10})

def test_dashboard_totals(self, client):
    client.post("/api/activity", json={"user_id": 1, "category": "electricity", "units":
20, ...})
    res = client.get("/api/dashboard/1")
    assert res.get_json()["totals"]["electricity"] == 16.4
```

5.3 Backend Test Execution

Run with: `pytest test_carbonsense.py -v`



```

backend - pytest -v
PS C:\Users\Lenovo\...\Carbonsense\backend> pytest test_carbonsense.py -v

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: backend/
collected 25 items

test_carbonsense.py::TestCarbonCalculator::test_travel_car_petrol PASSED [ 4%]
test_carbonsense.py::TestCarbonCalculator::test_travel_bike_is_zero PASSED [ 8%]
test_carbonsense.py::TestCarbonCalculator::test_electricity PASSED [ 12%]
test_carbonsense.py::TestCarbonCalculator::test_food_meat PASSED [ 16%]
test_carbonsense.py::TestCarbonCalculator::test_waste_composted PASSED [ 20%]
test_carbonsense.py::TestCarbonCalculator::test_invalid_category_raises PASSED [ 24%]
test_carbonsense.py::TestCarbonCalculator::test_invalid_travel_mode_raises PASSED [ 28%]
test_carbonsense.py::TestCarbonCalculator::test_negative_distance_raises PASSED [ 32%]
test_carbonsense.py::TestCarbonCalculator::test_non_numeric_distance_raises PASSED [ 36%]
test_carbonsense.py::TestCarbonCalculator::test_category_totals PASSED [ 40%]
test_carbonsense.py::TestRecommendation::test_empty_activities_returns_default_tip PASSED [ 44%]
test_carbonsense.py::TestRecommendation::test_high_car_km_triggers_carpool_tip PASSED [ 48%]
test_carbonsense.py::TestRecommendation::test_meat_heavy_diet_triggers_plant_based_tip PASSED [ 52%]
test_carbonsense.py::TestRecommendation::test_low_activity_does_not_overtrigger PASSED [ 56%]
test_carbonsense.py::TestRecommendation::test_returns_at_most_four_tips PASSED [ 60%]
test_carbonsense.py::TestAPI::test_health PASSED [ 64%]
test_carbonsense.py::TestAPI::test_login_creates_user PASSED [ 68%]
test_carbonsense.py::TestAPI::test_login_is_idempotent_by_email PASSED [ 72%]
test_carbonsense.py::TestAPI::test_login_missing_fields_returns_400 PASSED [ 76%]
test_carbonsense.py::TestAPI::test_add_activity_valid PASSED [ 80%]
test_carbonsense.py::TestAPI::test_add_activity_invalid_returns_400 PASSED [ 84%]
test_carbonsense.py::TestAPI::test_dashboard_totals PASSED [ 88%]
test_carbonsense.py::TestAPI::test_goal_set_and_get PASSED [ 92%]
test_carbonsense.py::TestAPI::test_goal_rejects_non_positive PASSED [ 96%]
test_carbonsense.py::TestAPI::test_recommendations_endpoint PASSED [100%]

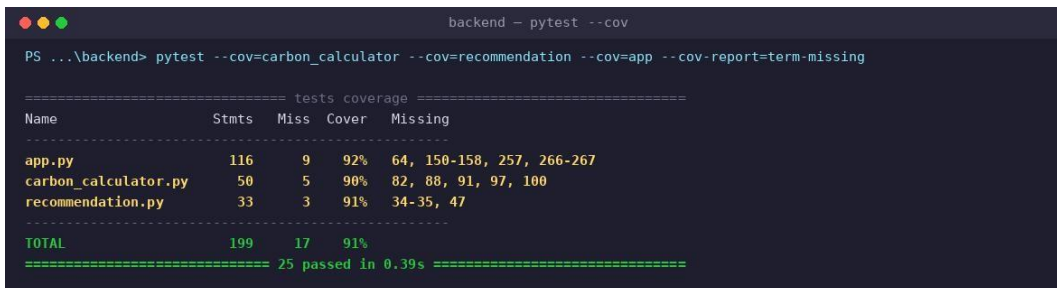
===== 25 passed in 0.35s =====

```

Figure 1: All 25 backend tests passing.

5.4 Code Coverage

Coverage was measured to confirm the test suite actually exercises the logic it claims to, not just that a small, convenient subset passes.



```

backend - pytest --cov
PS ...\\backend> pytest --cov=carbon_calculator --cov=recommendation --cov=app --cov-report=term-missing

===== tests coverage =====
Name                Stmts  Miss  Cover   Missing
-----
app.py                116     9    92%    64, 150-158, 257, 266-267
carbon_calculator.py  50      5    90%    82, 88, 91, 97, 100
recommendation.py     33      3    91%    34-35, 47

TOTAL                 199    17    91%

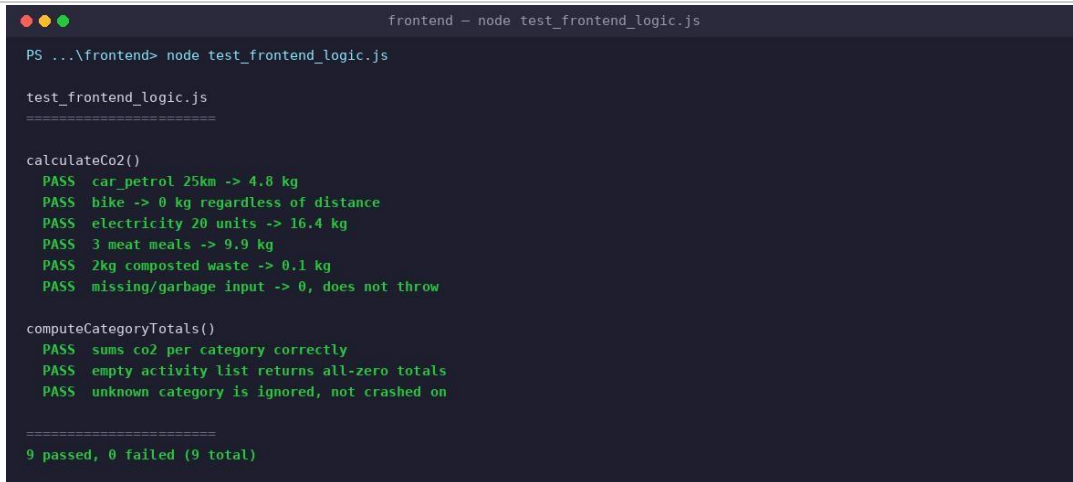
===== 25 passed in 0.39s =====

```

Figure 2: 91% combined statement coverage across the three backend modules.

5.5 Frontend Logic Test Execution

Run with: `node test_frontend_logic.js`



```

frontend -- node test_frontend_logic.js

PS ...\.frontend> node test_frontend_logic.js

test_frontend_logic.js

=====

calculateCo2()
PASS car_petrol 25km -> 4.8 kg
PASS bike -> 0 kg regardless of distance
PASS electricity 20 units -> 16.4 kg
PASS 3 meat meals -> 9.9 kg
PASS 2kg composted waste -> 0.1 kg
PASS missing/garbage input -> 0, does not throw

computeCategoryTotals()
PASS sums co2 per category correctly
PASS empty activity list returns all-zero totals
PASS unknown category is ignored, not crashed on

=====
9 passed, 0 failed (9 total)

```

Figure 3: All 9 frontend calculation-logic tests passing.

6. Repository Documentation

Documentation was assessed against four criteria: presence of a README, installation instructions, a usage guide, and a dependency list. Prior to this review, the repository had no README.md and no .gitignore — a genuine gap, reported here honestly rather than glossed over.

6.1 Documentation Completeness — Before and After

Item	Before this review	After this review
README.md	Absent.	Added: project description, tech stack, folder structure, run instructions (Docker and local), API table.
Installation instructions	Not documented anywhere in the repo.	Documented for both docker compose up --build and a manual pip install -r requirements.txt path.
Usage guide	Absent.	API endpoint table added, listing method, route, and purpose for all 8 routes.
Dependencies	Listed only implicitly in requirements.txt, with no explanation.	requirements.txt referenced directly from the README installation steps.
.gitignore	Absent — __pycache__ and similar artifacts were at risk of being committed.	Added, covering Python, editor, OS, and log artifacts.

6.2 In-Code Documentation

Separately from repository-level documentation, in-code documentation (docstrings and comments) was found to be strong in the backend and weaker in the frontend, as detailed in Section 3.3 — this is a code-quality observation rather than a repository-documentation gap, but the two compound each other: a missing README made the backend's docstrings the only source of truth for how to actually run the project.

6.3 Suggested Further Improvements

- **Add a Known Limitations / Roadmap section to the README:** so contributors can see, at a glance, what has and hasn't been implemented yet.
- **Add screenshots to the README:** a short GIF or 2–3 screenshots of the dashboard in the README would let a reviewer understand the app without cloning it.
- **Document the branching convention in-repo:** a CONTRIBUTING.md with branch-naming and commit-message conventions would make the Section 4 branching strategy explicit for new contributors rather than tribal knowledge.
- **Add API request/response examples:** the API table added to the README (Section 6.1) could be expanded into full request/response examples for each route.

7. Code Review Findings and Recommendations

7.1 Overall Findings

CarbonSense demonstrates solid fundamentals for a project of this scope: a cleanly modular backend, consistent naming conventions, real input validation, and a repository structure that mirrors the application's actual architecture. The most significant gaps found were the complete absence of both automated tests and repository-level documentation prior to this review — both of which have now been directly addressed as part of completing it (a 25-case backend test suite with 91% coverage, a 9-case frontend logic test suite, a README, and a .gitignore).

7.2 Summary of Recommendations

Area	Recommendation	Priority
Code quality	Remove the duplicated emission factors between <code>script.js</code> and <code>carbon_calculator.py</code> by serving them from one endpoint.	High — affects correctness if the two drift apart.
Code quality	Add type hints to <code>app.py</code> and <code>database.py</code> to match <code>carbon_calculator.py</code> .	Medium
Repository management	Split <code>script.js</code> into smaller modules (<code>api.js</code> , <code>render.js</code> , <code>state.js</code>) as the frontend grows.	Medium

Area	Recommendation	Priority
Documentation	Keep README.md up to date as new endpoints/features are added; add screenshots.	Medium
Testing	Wire the existing pytest suite into a CI pipeline (e.g. GitHub Actions) so it runs on every push automatically rather than manually.	High — currently the only thing preventing regressions is running tests by hand.
Future enhancement	Add a logging framework (e.g. Python's logging module) so production errors are visible rather than silently caught.	Medium

7.3 Closing Assessment

Taken together, the codebase, repository structure, and version-control history reflect a project that was planned before it was built rather than assembled ad hoc — the folder layout, naming conventions, and modular backend all point the same direction. The recommendations above are refinements to an already sound foundation, not corrections to a fundamentally disorganised one.